GRAPH ALGORITHMS

Dijkstra’s Algorithm

Single-Source Shortest Path Algorithm for Non-Negative Weighted Graphs

CORE PARADIGM
Greedy Approach

CREATOR
Edsger W. Dijkstra (1956)

OVERVIEW

Introduction to Dijkstra's Algorithm

Single-Source Purpose

Calculates the shortest distance from a starting node to all other nodes in a weighted graph.

Greedy Strategy

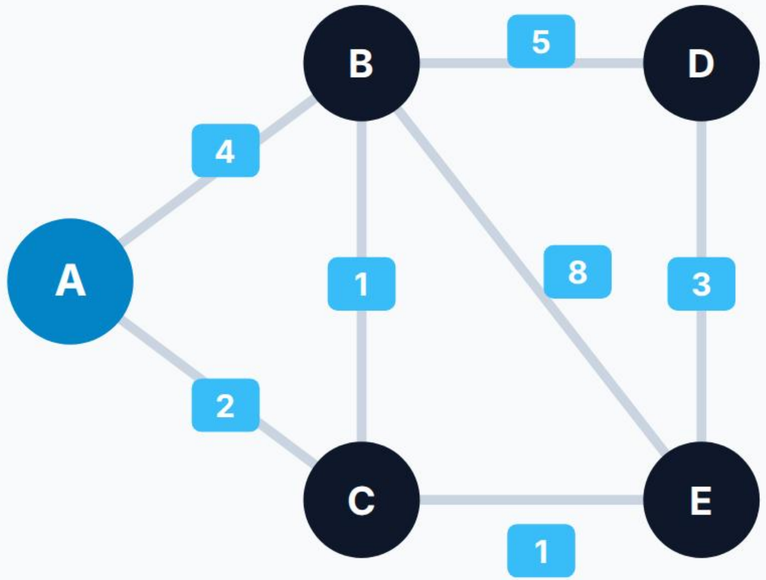
Selects the unvisited vertex with the minimum tentative distance at every step.

Key Constraint

Requires non-negative edge weights; negative weights lead to incorrect path decisions.

GRAPH MODEL

Sample Weighted Graph



Target Source

Node A (Start Vertex)

Initial State

Distance to A = 0; Distance to B, C, D, E = ∞

Goal

Find minimum path distances from Source A to all nodes.

EXECUTION WALKTHROUGH

Step-by-Step Calculation (Part 1)

STEP 1

Visit Node A (Dist = 0)

Relax neighbors of A:

```
Dist[B] = min(∞, 0 + 4) = 4
```

```
Dist[C] = min(∞, 0 + 2) = 2
```

Visited: {A} | Next Smallest: C (2)

STEP 2

Visit Node C (Dist = 2)

Relax neighbors of C (B, E):

```
Dist[B] = min(4, 2 + 1) = 3 (Updated)
```

```
Dist[E] = min(∞, 2 + 1) = 3
```

Visited: {A, C} | Next Smallest: B (3)

EXECUTION WALKTHROUGH

Step-by-Step Calculation (Part 2)

STEP 3

Visit Node B (Dist = 3)

Relax unvisited neighbors of B (D):

```
Dist[D] = min(∞, 3 + 5) = 8
```

```
Dist[E] = min(3, 3 + 8) = 3 (No change)
```

Visited: {A, C, B} | Next Smallest: E (3)

STEP 4

Visit Node E (Dist = 3)

Relax unvisited neighbors of E (D):

```
Dist[D] = min(8, 3 + 3) = 6 (Updated)
```

Visited: {A, C, B, E} | Final Node: D (6)

FINAL RESULTS

Shortest Path Matrix from Source (A)

Target Node	Shortest Distance	Optimal Path
Node A	0	A
Node B	3	A → C → B
Node C	2	A → C
Node E	3	A → C → E
Node D	6	A → C → E → D

Key Takeaway

Direct paths were overridden by shorter multi-hop routes. For example, **A → B** directly costs **4**, but routing through **A → C → B** reduced the total cost to **3**.

ALGORITHM DESIGN

Algorithm Pseudocode

```
function Dijkstra(Graph, source):
    for each vertex v in Graph:
        dist[v] = INFINITY
        visited[v] = false
    dist[source] = 0

    create Min-PriorityQueue Q
    Q.insert(source, 0)

    while Q is not empty:
        u = Q.extractMin()
        visited[u] = true

        for each neighbor v of u:
            if not visited[v] and dist[u] + weight(u, v) < dist[v]:
                dist[v] = dist[u] + weight(u, v)
                Q.decreaseKey(v, dist[v])
```

ANALYSIS & USE CASES

Complexity & Real-World Applications

Min-Heap / Priority Queue

$O((V + E) \log V)$

Optimal for sparse graphs where $E << V^2$

Array Implementation

$O(V^2)$

Simpler approach, best suited for dense graphs

Practical Applications

GPS Navigation
Google Maps & routing services for optimal roads.

Network Routing
OSPF & IS-IS protocols to route internet packets.

Game Development
AI pathfinding on grid maps for movement.

SUMMARY

Core Takeaways of Dijkstra's Algorithm

Efficient Greedy Logic

Guarantees optimal single-source shortest path for non-negative weights.

Foundation of Routing

Powers modern GPS navigation and core internet packet routing infrastructure.